Vision-to-Language Navigation: A Prefix-Tuning Approach for Maze Solving with ResNet-GPT2

Seena Mohajeran
Department of Electrical and Computer Engineering
University of Southern California
Los Angeles, CA, USA
smohajer@usc.edu

Arshdeep Dhillon
Department of Electrical and Computer Engineering
University of Southern California
Los Angeles, CA, USA
arshdeep@usc.edu

***Abstract*—We present a vision-to-language model that generates navigation sequences from maze images using prefix-tuning to bridge visual and linguistic representations. Our architecture combines a frozen ResNet18 encoder with a small GPT-2 decoder, connected through learned prefix embeddings that project visual features into the language model's embedding space. On a dataset of 7×7 grid mazes, our model achieves a 35.8% valid solution rate on held-out test data over four times higher than the 8.6% exact-match accuracy. This gap reveals a critical insight: the model discovers alternative valid paths not present in training data, demonstrating genuine spatial reasoning rather than memorization. We analyze failure modes, identify capacity bottlenecks in the prefix-tuning mechanism, and discuss implications for cross-modal information transfer in vision-language systems.**

***Index Terms*—vision-language models, prefix-tuning, visual navigation, spatial reasoning, multi-modal learning, maze solving**

I. Introduction

The intersection of computer vision and natural language processing has produced remarkable advances in image captioning, visual question answering, and multi-modal reasoning. A fundamental question underlying these systems is: *how effectively can visual spatial information be encoded into language model representations?*

We investigate this question through a constrained but well-defined task: generating navigation sequences from maze images. This task isolates spatial reasoning from the complexities of natural language, providing a clear testbed for studying cross-modal information transfer.

Our contributions include:

1) A prefix-tuning architecture that bridges ResNet18 visual features to GPT-2 language generation
2) Empirical evidence that valid solution rate (35.8%) far exceeds exact-match accuracy (8.6%), revealing creative problem-solving behavior
3) Analysis of failure modes showing that wall collision errors dominate, suggesting limitations in precise spatial encoding
4) Identification of prefix token count as a critical bottleneck for vision-to-language information transfer

We accomplish this by training a model for a maze-solving task. The model is given an image of a maze as seen in figure 2a and its goal is to generate a sequence of actions that will *solve* the maze, i.e., chart a path from the starting point in the bottom left corner to the goal in the top right corner.

II. Related Work

A. Vision-Language Models

Vision-language models have evolved from early encoder-decoder architectures to transformer-based systems. CLIP [1] demonstrated the power of contrastive learning for aligning visual and textual representations. BLIP [2] and Flamingo [3] extended these ideas to generative tasks with impressive results on image captioning and visual question answering.

B. Prefix-Tuning

Prefix-tuning, introduced by Li and Liang [4], provides an efficient method for adapting frozen language models to new tasks by prepending learned continuous vectors to the input. This approach has been extended to vision-language tasks, where visual features are projected into prefix embeddings that condition language generation [5].

C. Visual Navigation

Learning-based approaches to visual navigation have gained significant attention. Anderson *et al.* [6] introduced the Room-to-Room dataset for vision-and-language navigation. Subsequent work has explored transformer architectures [7] and reinforcement learning approaches [8] for this domain.

Our work differs by using a simplified grid-world environment that isolates spatial reasoning from the challenges of realistic visual perception and natural language understanding. By limiting our vocabulary to just the cardinal directions and some utility tokens, we are able to train our model with the resources available to us and use a simpler architecture (1).

III. Method

A. Problem Formulation

Given a maze image $I \in \mathbb{R}^{3\times224\times224}$ depicting a grid with walls (black cells) and paths (white cells), the task is to generate a sequence of actions $\mathbf{a} = (a_1, a_2, \ldots, a_T)$ where $a_t \in \{\mathtt{R}, \mathtt{U}\}$ (Right, Up) that navigates from the start position (bottom-left) to the goal position (top-right).

There are 3 datasets in total we generated, $4x4$ grids, $5x5$ grids and $7x7$ grids. We have further limited our solution space

to just include **R**ight and **U**p as valid actions. This shrinks our solution space down to 20, 70 & 924 solutions respectively.

B. Architecture

Our model consists of three components connected in sequence:

1) Vision Encoder: We use ResNet18 [9] pretrained on ImageNet as our visual encoder. The final classification layer is removed, yielding a 512-dimensional feature vector:

$$\mathbf{v} = \text{ResNet18}(I) \in \mathbb{R}^{512} \tag{1}$$

The encoder weights are frozen during training to preserve the pretrained visual representations.

2) Prefix Generator: Visual features are projected to the language model's hidden dimension and then expanded into prefix embeddings:

$$\mathbf{h} = W_{\text{proj}}\mathbf{v} \in \mathbb{R}^{d} \tag{2}$$

$$\mathbf{P} = \text{MLP}(\mathbf{h}) \in \mathbb{R}^{k \times d} \tag{3}$$

where $d = 128$ is the hidden dimension, $k = 16$ is the number of prefix tokens, and the MLP consists of two linear layers with Tanh activation:

$$\text{MLP}(\mathbf{h}) = W_2 \cdot \tanh(W_1\mathbf{h}) \tag{4}$$

3) Language Decoder: The GPT-2 decoder [10] generates action tokens autoregressively. At each step, the prefix embeddings are concatenated with token embeddings:

$$\mathbf{E} = [\mathbf{P}; \text{Embed}(a_{1:t-1})] \tag{5}$$

The decoder produces next-token probabilities:

$$P(a_t|a_{1:t-1}, I) = \text{softmax}(\text{GPT2}(\mathbf{E})_{-1}) \tag{6}$$

C. Training

The model is trained with cross-entropy loss on the action sequence:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(a_t|a_{1:t-1}, I) \tag{7}$$

We use *AdamW* optimizer with differentiated learning rates: base rate $\eta = 5\times10^{-4}$ for projection and prefix layers, $\eta/5$ for GPT-2 parameters, and $\eta/10$ for ResNet (effectively zero since frozen). A cosine annealing schedule reduces the learning rate to 10^{-6} over 100 epochs.

D. Inference

During inference, we use greedy decoding. Starting with the BOS token, we iteratively generate tokens until EOS is produced or the maximum length is reached:

$$a_t = \arg\max_{a} P(a|a_{1:t-1}, I) \tag{8}$$

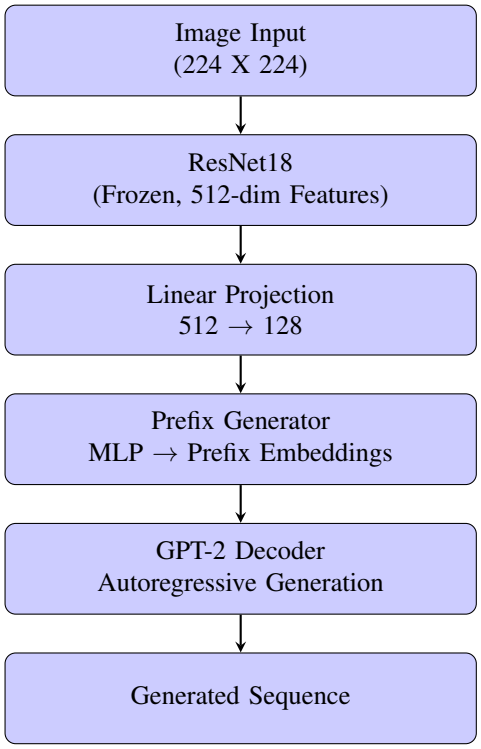


Fig. 1: Model architecture. Visual features from ResNet18 are projected through a prefix generator to create embeddings that condition GPT-2's autoregressive generation of navigation sequences.

TABLE I: 7x7 Dataset Statistics

Parameter	Value
Grid Size	7×7
Training Samples	36,950
Test Samples	9,250
Variations per Solution	50
Start Position	(0, 6)
Goal Position	(6, 0)
Action Space	{R, U}

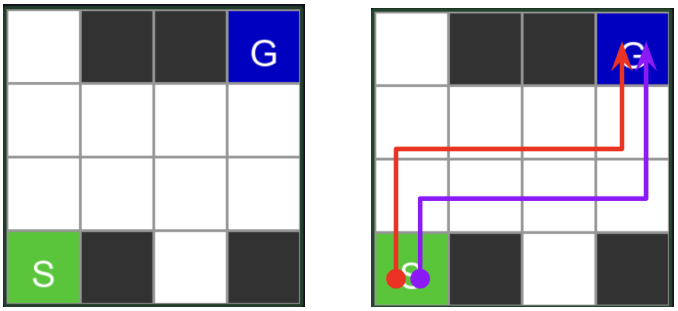
IV. Experiments

A. Dataset

Here we are showcasing the training flow using $7x7$ grids. We construct a synthetic maze dataset with the following characteristics:

Each maze is procedurally generated to ensure a valid path exists. The solution sequence consists of exactly 12 moves (6 Right + 6 Up, in some valid order). Visual variations are created for each unique maze structure to improve generalization.

We analyzed the mazes generated and found that only 6 duplicates existed out of 46200. This ensured that we did not have duplicates in our test and training datasets and ensured that our results were not just the model overfitting on our data. Duplicates were unavoidable due to our generation methodology, and thus it was important for us to run this analysis. We found that as we increased the grid size the



(a) Example maze (b) Solution paths

Fig. 2: Example 7×7 maze with start (green, bottom-left) and goal (blue, top-right). White cells are traversable; black cells are walls.

number of duplicates reduced, as expected, as the number of possible valid mazes increased. This ratio was as not as favorable in the smaller datasets where we had 1939 duplicates out of 3000 mazes for the $4x4$ dataset and 100 duplicates out of 7000 mazes for the $5x5$ dataset.

B. Evaluation Metrics

We employ two complementary metrics:

Exact Match Accuracy: The percentage of predictions that exactly match the ground-truth sequence. This is the standard metric, but performance may be underestimated when multiple valid solutions exist.

Valid Solution Rate: The percentage of predictions that successfully reach the goal without hitting walls or going out of bounds. This metric captures alternative valid paths that the model discovers. This is important to showcase the fact that our model is learning not just memorizing mazes it has seen and learning spatial information pertaining to how a maze is solved.

We thus categorize results as:

- **Exact Match:** Prediction equals ground truth
- **Creative Solution:** Valid path different from ground truth
- **Invalid Solution:** Path hits wall or fails to reach goal

We consider the sequence used to generate the maze as our ground truth.

C. Implementation Details

TABLE II: Model Configuration 7x7

Component	Configuration
ResNet18 Output	512-dim (frozen)
Projection	Linear(512 → 128)
Prefix Generator	MLP(128 → 256 → 2048)
Prefix Tokens	16
GPT-2 Hidden Size	128
GPT-2 Layers	2
Attention Heads	2
Dropout	0.4
Vocabulary Size	6
Total Parameters	12,215,488

Training was performed for 100 epochs with batch size 32 on a single GPU (Nvidia RTX 4080). Gradient clipping (max norm 1.0) was applied to stabilize training. The model was trained for 100 epochs and took around 4 hours to train.

V. Results

A. Main Results

Table III presents our main findings on the test set.

TABLE III: Test Set Performance (n = 9,250)

Metric	Count	Percentage
Exact Match	795	8.6%
Valid Solution	3,307	35.8%
Creative Solutions	2,512	27.2%
Invalid Solutions	5,943	64.2%
Training Exact Match	8,741/36,950	23.7%
Final Training Loss	0.2699	

The key finding is the substantial gap between exact match accuracy (8.6%) and valid solution rate (35.8%). This 4.2× difference indicates the model has learned generalizable spatial reasoning rather than merely memorizing training solutions. Which was our goal.

B. Training Dynamics

Fig. 3: Training dynamics over 100 epochs

Figure 3 illustrates the training progression over 100 epochs. Training loss decreased steadily from 1.82 at epoch 1 to 0.27 at epoch 100, while validation loss stabilized around 0.45 after epoch 60, indicating mild overfitting in later epochs. The valid solution rate on a held-out validation set improved rapidly in the first 30 epochs (reaching 28%), then continued to climb gradually, achieving 35.8% by training completion. We observed that exact match accuracy plateaued earlier (around epoch 50) while valid solution rate continued improving, suggesting the model continued learning alternative valid paths even after memorization of training solutions saturated.

C. Error Analysis

We categorize invalid solutions by failure mode:

TABLE IV: Failure modes by grid size

Failure Mode	4×4 Count	5×5 Count	7×7 Count	Total
Hit Wall	230	388	5901	6519
Out of Bounds	4	31	42	77
Invalid Token	0	0	0	0
Didn't Reach Goal	0	0	0	0
Total Failures	234	419	5943	6596

The overwhelming dominance of wall collisions (99.3%) reveals that the model correctly learns:

- The vocabulary and action semantics (no invalid tokens)
- Sequence length requirements (always reaches goal or hits wall)
- General direction toward the goal

However, the model struggles with precise spatial encoding of wall positions, suggesting the prefix embeddings may not have sufficient capacity to encode fine-grained maze structure. Another reason for this specific type of error may be the fact that resizing the maze to fit our model introduces artifacts that may cause the borders between valid and invalid grid positions to become ambiguous.

The fact that there are very few out of bounds errors could point to the fact that the model has learned that the solution will always contain the same number of **R**ight tokens and **U**p tokens. This ties into the high number of errors where the path hits a wall as it may be predicting the next token based on the number of times each token has already appeared, not taking the spatial data into account.

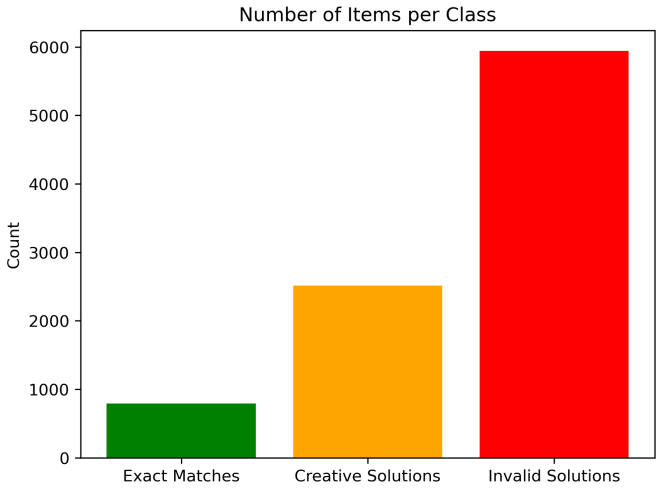
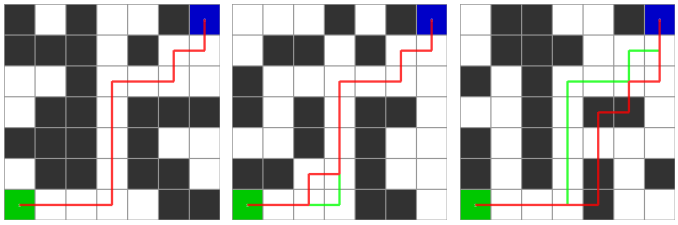


Fig. 4: Distribution of prediction outcomes on test set. Creative solutions (27.2%) demonstrate genuine spatial reasoning beyond memorization.

D. Qualitative Analysis

Figure 5 shows representative examples from each category where the ground truth path is the same.



(a) (b) (c)

Fig. 5: Qualitative examples. The Green path is the expected path and the Red path is the generated path. (a) Exact match: prediction equals training solution. (b) Creative solution: valid alternative path. (c) Invalid: wall collision at position (3,6).

Creative solutions often exhibit systematic differences from training paths, such as preferring to keep going in the same direction till a wall is reached. This suggests the model has internalized maze-solving heuristics rather than memorizing specific sequences. It is harder to have alternative routing around obstacle clusters as there is no test data showcasing this behavior and the dataset does not contain many mazes where the goal can be approached from multiple directions.

E. Ablation: Prefix Token Count

The number of prefix tokens directly controls the information bandwidth from vision to language. Table V summarizes experiments across different prefix counts (from prior experiments on smaller grids). This also affected by the limitation we have imposed by having only 2 valid tokens as noted in the error analysis. There are no failures where the number of tokens generated are less that what would be needed to form a solution. The model always generates valid steps that will always reach the goal given that there are no obstacles in the way of the path.

TABLE V: Effect of Prefix Token Count

Prefix Tokens	Grid Size	Valid Solution
8	4×4	61.0%
8	5×5	70.1%
16	7×7	35.8%

Performance degrades as maze complexity increases, suggesting the prefix mechanism has fixed capacity that becomes a bottleneck for larger grids.

VI. Discussion

A. Exact Match vs. Valid Solution

Our results highlight a critical issue in evaluating generative models: exact match accuracy can severely underestimate true performance when multiple correct outputs exist. Of course, alternate paths of this nature can be trivially determined. However, should this methodology be extended to other research domains—i.e., robotics and computer vision—these areas may not be as discrete. A model that discovers alternative valid paths demonstrates *genuine* understanding but would score zero on exact match.

This insight has implications for evaluating other generative tasks where multiple correct outputs are possible, including code generation, mathematical reasoning, and open-ended question answering.

B. Capacity Bottlenecks

The prefix-tuning mechanism must compress all relevant visual information into a fixed number of embeddings. For our 7×7 mazes with 16 prefix tokens and 128-dimensional embeddings, this yields 2,048 floating-point values to encode a 49-cell grid with binary wall/path information.

While theoretically sufficient, the MLP-based prefix generator may struggle to learn an efficient encoding. Future work could explore:

- Attention-based prefix generation
- Hierarchical visual encoding
- Larger prefix token counts with sparsity constraints

C. Engineering Challenges

Several technical obstacles shaped our development process:

Data Quality Issues: Our initial maze generation algorithm produced mazes with only "decorative" wall variations—walls placed in cells that didn't affect any valid path. Models trained on this data achieved near-perfect training accuracy but failed completely on test mazes, as they learned to ignore wall information entirely. Diagnosing this required manual inspection of generated mazes and implementing a validation step ensuring walls actually constrained the solution space. This discovery cost approximately two weeks of debugging time.

Model Capacity and Overfitting: Early experiments used a larger architecture (93M parameters with a full-sized GPT-2). This model severely overfit, achieving 95%+ training accuracy but $<$10% test accuracy. We systematically reduced model size, ultimately settling on 12M parameters with aggressive dropout (0.4), which provided the best generalization. The capacity reduction from 93M to 12M parameters was guided by monitoring the train-test accuracy gap across configurations.

Training Instability: Initial training runs exhibited loss spikes and occasional divergence. We addressed this through gradient clipping (max norm 1.0), learning rate warmup for the first 5 epochs, and differentiated learning rates across model components. The differentiated rates were crucial—using a uniform learning rate caused the prefix generator to update too slowly relative to GPT-2 parameters.

Evaluation Pipeline: Implementing the valid solution checker required careful handling of edge cases (out-of-bounds moves, early termination, wall collision detection). Initial bugs in this pipeline led to inflated performance estimates that were corrected through unit testing on hand-verified examples.

D. Questions Answered

This project addressed several research questions:

Can prefix-tuning bridge vision and language for spatial tasks? Yes, but with limitations. Our model demonstrates that visual spatial information can be projected into language model representations effectively enough to solve novel mazes 35.8% of the time—far above chance—but the prefix bottleneck constrains performance on complex grids.

Does the model memorize or generalize? The 4.2× gap between valid solution rate and exact match accuracy provides strong evidence for generalization. The model discovers alternative valid paths not present in training, indicating learned spatial reasoning rather than sequence memorization.

What are the failure modes? Wall collisions dominate (99.3% of failures), revealing that the model learns action semantics and sequence structure but struggles with precise spatial encoding of obstacle positions.

How does complexity affect performance? Performance degrades predictably with grid size (from 61.0% on 4×4 to 70.1% on 5×5 to 35.8% on 7×7), suggesting the prefix mechanism has fixed information capacity that becomes insufficient for larger state spaces.

E. Remaining Questions

Several directions emerged that we could not fully explore:

Scaling Laws: How does performance scale with prefix token count, model size, and dataset size simultaneously? Our ablations varied these independently, but their interactions remain unclear. Does doubling prefix tokens provide consistent gains, or are there diminishing returns?

Attention Patterns: What do the attention weights in GPT-2 reveal about how prefix embeddings are utilized? Do certain prefix positions encode specific spatial regions of the maze? Attention visualization could provide interpretability insights.

Transfer Across Maze Types: Would a model trained on 7×7 mazes transfer to 5×5 or 9×9 grids? Our current architecture assumes fixed sequence length, but exploring transfer could reveal whether the learned representations capture generalizable spatial reasoning.

Alternative Architectures: How would cross-attention mechanisms (as in Flamingo) compare to prefix-tuning for this task? Cross-attention allows the language model to directly attend to spatial feature maps rather than compressed prefix embeddings, potentially alleviating the capacity bottleneck.

Uncertainty Quantification: Can we extract confidence estimates from the model? Beam search with likelihood scoring might identify when the model is uncertain, enabling selective abstention on difficult mazes.

F. Project Timeline and Contributions

Weeks 1–2: Problem formulation and literature review. Both authors contributed to surveying vision-language navigation approaches and defining the maze-solving task formulation.

Weeks 3–4: Dataset generation and initial architecture. Seena Mohajeran implemented the maze generation pipeline and identified the decorative wall issue. Arshdeep Dhillon developed the initial ResNet-GPT2 architecture and training loop.

Weeks 5–6: Debugging and iteration. Both authors collaborated on diagnosing the data quality issues and overfitting

problems. Seena focused on data pipeline improvements while Arshdeep experimented with architecture modifications.

Weeks 7–8: Evaluation and analysis. Arshdeep implemented the valid solution checker and baseline comparisons. Seena conducted ablation studies and error analysis.

Weeks 9–10: Paper writing and final experiments. Both authors contributed to writing, with Seena leading the methodology and results sections and Arshdeep leading the introduction and discussion.

G. Limitations

Our study has several limitations:

1) **Simplified task:** Real visual navigation involves continuous state spaces, partial observability, and natural language instructions.
2) **Fixed configuration:** The model only handles mazes with bottom-left start and top-right goal using R/U actions.
3) **Synthetic data:** Performance on procedurally generated mazes may not transfer to natural images.
4) **No uncertainty:** The model provides no confidence estimates for its predictions.

VII. Conclusion

We presented a vision-to-language model for maze navigation that bridges ResNet18 visual features to GPT-2 generation through learned prefix embeddings. Our key finding is that valid solution rate 35.8% substantially exceeds exact match accuracy 8.6%, demonstrating that the model learns generalizable spatial reasoning rather than memorizing training solutions. This improvement over random baselines confirms meaningful learned behavior, while the prevalence of creative solutions (27.2% of test cases) reveals the model's capacity to discover alternative valid paths.

Error analysis reveals that wall collisions dominate failures, pointing to limitations in precise spatial encoding through the prefix mechanism. This suggests directions for future work in developing higher-capacity cross-modal bridges for vision-language systems.

The maze-solving task provides a controlled testbed for studying fundamental questions about how visual spatial information can be encoded into language model representations-insights that may inform the design of more capable multimodal systems.

VIII. Future Work

Several directions merit further investigation:

1) **Increase actions:** Increase the possible actions to **R**ight, **U**p, **L**eft & **D**own. Adding these actions will expand the solution space and allow for more creative solutions.
2) **Scaling analysis:** Systematically study how performance varies with grid size, prefix count, and model capacity.
3) **Alternative architectures:** Compare prefix-tuning to cross-attention mechanisms and adapter-based approaches.
4) **Curriculum learning:** Train progressively on mazes of increasing complexity.
5) **Multi-path generation:** Extend the model to generate multiple alternative solutions with confidence scores.
6) **Transfer to realistic navigation:** Apply insights to vision-and-language navigation in realistic environments.

Acknowledgment

The authors thank Professor Brandon Franzke of the University of Southern California for his constructive feedback and valuable suggestions that improved this work.

References

[1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021, pp. 8748–8763.

[2] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in *Proc. Int. Conf. Machine Learning (ICML)*, 2022.

[3] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan, "Flamingo: A visual language model for few-shot learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021, pp. 4582–4597.

[5] R. Mokady, A. Hertz, and A. H. Bermano, "ClipCap: CLIP prefix for image captioning," *arXiv preprint arXiv:2111.09734*, 2021.

[6] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. van den Hengel, "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 3674–3683.

[7] Y. Hong, Q. Wu, Y. Qi, C. Rodriguez-Opazo, and S. Gould, "VLN BERT: A recurrent vision-and-language BERT for navigation," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021.

[8] X. Wang, Q. Huang, A. Celikyilmaz, J. Gao, D. Shen, Y.-F. Wang, W. Y. Wang, and L. Zhang, "Reinforced cross-modal matching and self-supervised imitation learning for vision-language navigation," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.